Hit Rate Is Not Output Quality: Characterizing KV-Cache Reuse on Agent Traffic

A Replay Study of Approximate KV Reuse on Coding-Agent Traffic

Yiheng "Intel" Chen
University of Pennsylvania
Philadelphia, PA, USA
intel@intelchen.com

Abstract

Modern LLM agent harnesses such as Claude Code generate long multi-turn sessions where the conversation routinely exceeds the model's context window, triggering `/compact`-style summarization that resets most of the KV cache. *Cacheblend* [16] proposes selective KV recompute to bridge this "compaction cliff," but published evaluations have focused on retrieval-augmented generation rather than agent traffic, and have measured byte-level *hit rate* rather than *output quality*.

We present skillcacher, a transparent proxy (drop-in for Claude Code clients, relying on a patched lmcache backend) that integrates cacheblend with real Claude Code traffic through three small but load-bearing patches: a structural-anchor segment parser that injects cacheblend's chunk separators around CC-shaped blocks; a chunk-aligned `LOOKUP` fix that resolves an upstream lmcache/vllm hash mismatch on chunk boundaries; and per-turn header normalization that stabilizes chunk hashes across multi-turn sessions. We additionally publish ClaudeCodeTrace, a public benchmark artifact of 13 redacted Claude Code captures spanning 182 turns and 411 k tokens across three subsets: `swebench_verified/` (5 captures, 24 turns), `post_compact/` (7 captures, 105 turns), and `skill_invocation/` (1 capture, 53 turns).

The evaluation in this paper is structured around three nested denominators: **the main corpus** (combined n=99 substantive turns, SWE-V ∪ `post_compact` v6+v7 ∪ `skill_invocation`; used for rescue rate + output identity), **the deep-evaluation subset** (SWE-V ∪ `post_compact` v6+v7, n_{sub}=47; used for TTFT, tool-call mode, recompute-ratio sweep, and LLM-judge), and **the divergent-judged slice** (n_{div}=19 of the 47, the non-bit-identical pairs).

On Llama-3.3-70B-Instruct fp8 with our patches, across the main n=99 corpus cacheblend rescues a *median* 76.5% of substantive-turn prefill tokens (p25 67%, p75 95%); 6% of turns reach the published 95–99% steady-state peak, but only on long-prompt workloads — on `skill_invocation/` the peak vanishes entirely (0/52 turns ≥99%). To our knowledge this is the first measurement of cacheblend on naturalistic Claude-Code-shaped traffic. On the n_{sub}=47 deep-evaluation subset, two further costs emerge that the published hit-rate framing elides. *Latency:* cacheblend's median TTFT is 55% larger than the no-cache baseline and 7× larger at the ≥99% rescue peak, so high rescue does not translate into latency reduction on this setup. *Agent-protocol usefulness:* 30% of substantive turns at T=0 produce severely divergent output (<0.3 token-identity to no-cache), including 11% with decode-mode flips (tool-call → prose); on the divergent slice a Sonnet-4.6 LLM-judge blind A/B prefers cacheblend on only 32% of pairs (95% Wilson CI [15%, 54%]), with cacheblend conditionally non-inferior on 63% vs. an 85% pre-registered gate. We argue that cacheblend's published hit-rate numbers, while real, come with both a latency cost and an agent-protocol-usefulness cost that systems papers should report alongside hit rate.

Keywords

LLM serving, KV cache, prefix caching, cacheblend, agent systems, Claude Code

1 Introduction

Large language model serving has converged on a layered KV-cache architecture: vLLM [11] provides paged-attention prefix caching within a session, and LMCache [14] extends it to a cross-request CPU-resident store. These mechanisms cleanly handle the common case where requests share an exact textual prefix (retrieval-augmented generation, fixed system prompts, etc.). They *cannot* handle the case where two requests share *most* of their tokens but differ in a chunk near the front — the textbook cacheblend [16] use case — without recomputing the entire suffix.

Modern agent harnesses sit precisely in that gap. Claude Code [1] (CC), the deployment we study, drives an LLM through long multi-turn task sessions: a single SWE-bench Verified task averages 5 turns at 5 k–10 k prompt tokens each, and a typical half-day session routinely crosses 100 k tokens before triggering CC's built-in `/compact` summarization. `/compact` replaces most of the conversation history with a roughly 800-token summary — which, by construction, breaks the prefix-match invariant that prefix caching depends on. *Every* post-compaction request re-prefills nearly from scratch on existing caches.

Cacheblend's promise and its production gap. Cacheblend [16] proposes a *selective KV recompute* strategy: chunk a request's tokens by a special separator, retrieve cached KV blocks for the chunks that match, and recompute KV only for a configurable fraction (default 15%) of the chunks that have shifted positionally. The published evaluation shows 70–90 % hit rates on permuted RAG retrieval workloads, where chunked retrieval is the design pattern users already write toward. Adapting cacheblend to CC traffic, however, exposes three integration issues that prior work does not address:

(1) **Chunk-boundary mismatch.** On lmcache v0.4.2 against vLLM v0.19.0, the `STORE` path hashes a chunk-aligned token range ($\lfloor n/256 \rfloor \cdot 256$) while the `LOOKUP` path hashes the full prompt range (n). Store and lookup hashes never agree, retrieval rate is 0% regardless of how the workload is shaped.

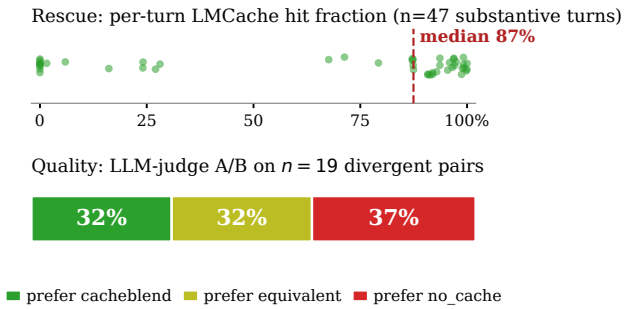

Figure 1: skillcacher’s headline finding on Llama-3.3-70B-Instruct fp8. Cacheblend rescues 95–99% of post-compaction prompt tokens at the prefill stage (top), reproducing the published rescue numbers on natural Claude Code agent traffic (to our knowledge for the first time on this workload class). But on the divergent (capture, turn) pairs — the 19 of 47 substantive pairs in the deep-evaluation subset where cacheblend’s blended-KV output is not bit-identical to a no-cache baseline at T=0 — a Sonnet-4.6 LLM judge prefers cacheblend’s output on only 31.6% of pairs and finds it non-inferior on 63.2% (bottom), well below the design specification’s 85% gate. Hit rate is not output quality.

(2) **Per-turn hash drift.** CC’s `x-anthropic-billing-header` embeds per-turn fields (`cch=`, a chunk hash; `cc_version=`, the CC binary version) into the first KV chunk. Within a single multi-turn session, these change between turns, so chunk 0’s hash is distinct on every request — cache hits do not propagate even when the rest of the prompt is identical.

(3) **No structural separators.** Cacheblend’s segment detector requires its configured `blend_special_str` (default `# #`) to appear at chunk boundaries. CC’s outbound traffic uses `\n\n`, which tokenizes to 25–180 tokens per segment — below cacheblend’s 256-token `blend_min_tokens` floor. The segment detector finds zero chunks; cacheblend never engages.

The output-quality question. A second gap is methodological. Cacheblend’s published numbers are expressed in cache-byte hit rate. But the system’s value proposition ultimately rests on the assumption that the rescued KV produces *the same output* as the all-fresh-compute baseline. Selective KV recompute is, by construction, an approximation: the recomputed chunks see different positional and attention context than the original compute pass. The cacheblend paper [16] notes “small but bounded quality drift” but does not quantify it on agent workloads, where a single-token difference between a tool-call JSON block and natural-language prose can be the difference between agent progress and a stalled task.

Contributions. This paper makes three contributions:

(1) **A working integration of cacheblend with real Claude Code traffic** (§3). We present three patches addressing the integration issues above and show, on a Llama-3.3-70B-Instruct fp8 backend with tensor-parallelism, a median 76.5% rescue rate over n=99 substantive turns across two ClaudeCodeTrace subsets, with a 6% mass of turns achieving the published 95–99 % steady-state peak on long-prompt SWE-V traffic and 0% on the `skill_invocation/` workload.

(2) **ClaudeCodeTrace, a reproducible public benchmark** (§4) of 13 redacted real Claude Code captures spanning 182 turns and 411 k total tokens across 5 SWE-bench Verified tasks, a skill-invocation capture covering 5 skills $\times$ 3 prompts, and 7 post-`/compact` multi-turn diagnostic captures, published under CC-BY 4.0 on Hugging Face [3].

(3) **To our knowledge, the first measurement of cacheblend’s agent-protocol usefulness cost on natural agent traffic** (§5). At $T = 0$, cacheblend produces bit-identical output to no_cache on 57% of substantive turns and severe (<0.3 token-identity) divergence on 30%, including 11% with decode-mode flips. A Sonnet-4.6 LLM-judge blind A/B over the divergent pairs prefers cacheblend on 32% (95% Wilson CI [15%, 54%]) and finds cacheblend conditionally non-inferior on 63% (end-to-end non-inferiority across all substantive turns is 85%, just clearing the 85% spec gate when bit-identical pairs are counted as guaranteed equivalents).

We close with a discussion of what this means for production deployment (§6): cacheblend’s rescue is real but not free, and systems papers should report quality alongside hit rate.

2 Background

This section establishes the four pieces of vocabulary that §3–§5 assume: vLLM’s paged-attention prefix cache, LMCache’s cross-request KV store, cacheblend’s selective recompute, and Claude Code’s `/compact` boundary. The `/compact` boundary (§2.4) is the integration target.

2.1 vLLM and prefix caching

vLLM [11] serves a Transformer LLM by paging its KV cache into fixed-size blocks (typically 16 tokens). Each block is identified by a content hash over the token IDs that produced it; when an incoming request’s tokenized prompt has a prefix whose content hashes match blocks already resident in the cache, vLLM skips the prefill compute for those blocks and reuses the cached KV directly. Because the hash is over the exact token sequence and the model weights are fixed, prefix caching is mathematically lossless: the KV state at the end of a cache-hit prefill is bit-identical to the state that fresh recomputation would produce.

This guarantee — same prefix, same weights, same KV — is what makes our no_cache↔prefix_cache equivalence result in §5.3 trivially expected at $T = 0$, and what makes the cross-request mechanism in §2.2 possible.

2.2 LMCache and cross-request KV reuse

vLLM’s prefix cache lives inside a single engine instance and evicts under GPU memory pressure. LMCache [14] extends the prefix-cache substrate by adding a cross-request, CPU- and disk-backed

store. The lookup semantics are the same (content-addressed by chunk hash), but the working set is two to three orders of magnitude larger and persists across requests, sessions, and pod restarts.

We use the V1 connector (`LMCacheConnectorV1` in vLLM 0.19+) and its chunk granularity (256 tokens) throughout. Each chunk that matches the cross-request store contributes to the request's *cache_read tokens* counter, which is the per-turn measurement of `lmcache`'s engagement with that request — we report it directly in §5.1 as the rescue rate.

2.3 Cacheblend's selective recompute

Plain LMCache requires *exact prefix match*: a single token inserted at position k invalidates the cache for every chunk from k onwards. This is fine for cache-friendly workloads (RAG with stable documents, frozen system prompts), but it loses against any workload that shifts content positionally — and post-compaction agent traffic (§2.4) is exactly that.

Cacheblend [16] relaxes the exact-prefix requirement to *positional* match: when a chunk that was the 5th in one request appears as the 3rd in another, cacheblend reuses the cached KV for the chunk but recomputes a small fraction of the attention layers in the rotated positional context (the *selective recompute*). The relevant knobs:

- `blend_min_tokens` (256 default): minimum chunk size at which the blend mechanism engages. Sub-256-token requests bypass cacheblend and fall through to plain prefix caching.
- `blend_recompute_ratios` (0.15 default): fraction of attention layers recomputed per blended chunk. Higher means less quality drift but more compute — the entire point of the mechanism is that 0.15 is supposed to be enough.
- `blend_special_str` (`# #` default): the string sentinel that marks chunk boundaries in the prompt.

The published cacheblend evaluation [16] reports 70–90% rescue on permuted-RAG workloads and notes "small but bounded" quality drift, citing accuracy preservation on TruthfulQA at a single recompute ratio. Agent traffic is not RAG; at $T = 0$ greedy decoding single-token differences are load-bearing; hit rate is not output quality. §5 measures all three.

2.4 Claude Code and the `/compact` cliff

Claude Code [1] is a coding agent that drives a Claude model (or, through our proxy, any LLM exposing an Anthropic-shaped API) through multi-turn tasks. Each turn's request body is, roughly, the concatenation of: a ~2.5k-token system prompt; the accumulated conversation history (user messages, model responses, tool calls, and tool outputs); the user's latest message; and a list of available tools. As the conversation grows turn over turn, the prompt grows strictly: turn 5 of a session contains all the tokens of turn 4 plus the new turn's input and output.

This monotone-growth structure is, by construction, the case prefix caching is designed for. The pre-compact phase of a CC session exhibits near-perfect prefix-cache utilization in practice — each turn re-prefills only the new tokens of that turn.

The discontinuity is `/compact`. At a configurable threshold (default ~80% of the context window, with a hard floor at 100k tokens), CC asks the LLM to summarize the conversation so far into an 800-token, 9-section structured summary. The next request then sends: system prompt + the 800-token summary + a placeholder "conversation continues" marker + the latest user message. The pre-compaction history is gone; in its place is a synthesized summary that the model has never seen at this prefix position before.

This is the cliff. From the prefix cache's perspective, the post-compaction request shares only its system prompt with the pre-compaction session — everything after the first ~2.5k tokens is new. The prefix cache hit ratio collapses to roughly `(system_prompt_tokens / total_prompt_tokens)`, which on a typical 30k-token post-compact request is around 8%. The remaining ~27.5k tokens must be reprefilled, paying a latency hit that scales linearly with prompt size and a recurring compute cost on every post-compact turn for the rest of the session.

Cacheblend was designed for this pattern — summary insertion is exactly the positional-shift case its selective recompute targets — but, as §3 shows, adapting cacheblend to real CC traffic required closing three concrete integration gaps before any of the rescue numbers in §5.1 held in practice.

3 System Design

skillcacher is a drop-in proxy that sits between Claude Code (CC) and a vLLM backend serving Llama-3.3-70B-Instruct fp8 with LMCache's `LMCacheConnectorV1` attached for cross-request KV reuse. Figure 2 shows the data path. CC issues Anthropic-shaped requests against the proxy's `/v1/messages` endpoint; the proxy translates each request to vLLM's OpenAI-compatible `/v1/chat/completions` API, applies three outbound transformations, and forwards. The cacheblend integration we present in this section consists of those three transformations (§3.1–§3.2.2); an offline statistical-span miner (§3.3) ships with the artifact but is not exercised in this paper's measurements.

3.1 CC-segment parser

The chunking problem. Cacheblend's segment detector expects its configured `blend_special_str` (default `# #`) to appear at chunk boundaries in the prompt. Chunks between separators are hashed and stored individually; chunks below `blend_min_tokens` (default 256) are ignored. CC's outbound traffic uses `\n\n` as its predominant block delimiter, which produces 25–180-token segments on the Llama tokenizer — below the 256-token floor. Against natural CC traffic with the published default, the segment detector found zero chunks and cacheblend's retrieval rate was 0% regardless of the workload's textual repetition.

Structural anchors. The parser identifies semantically meaningful CC block boundaries by literal anchor matching, then injects the cacheblend separator on either side of each recognized block. Anchors are deliberately content-stable across CC versions — they key on substrings like `Status:\n` (gitStatus preamble), `Recent commits:\n`, the `<system-reminder>`/`</system-reminder>` wrapper, `Summary:\n1.` (the 9-section auto-compact summary), and tool-output markers — rather than on version digits or content hashes that drift between releases.

Table 1 lists the recognized block types, their anchors, and typical token lengths on Llama-3 tokenization. The parser walks each

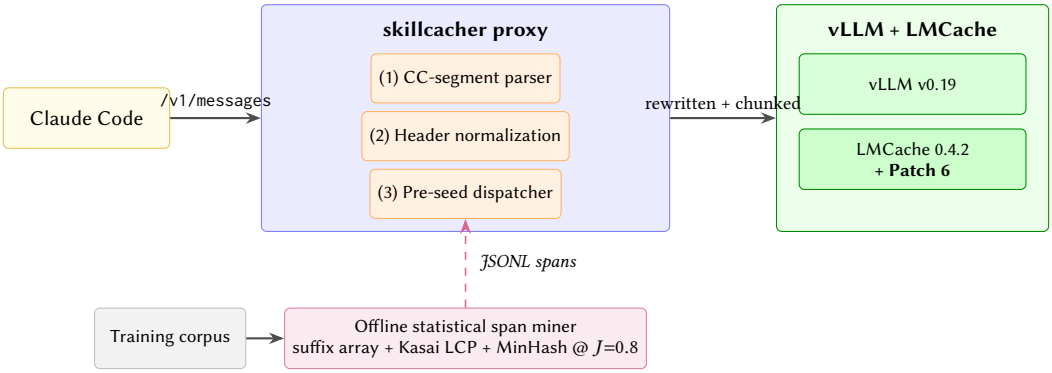

Figure 2: skillcacher architecture. The proxy applies three transformations to every outbound request: a CC-segment parser (§3.1) that injects cacheblend's separator around recognized structural blocks; per-turn header normalization (§3.2.2) that stabilizes the first KV chunk's hash across turns; and a pre-seed dispatcher driven by an offline statistical span miner (§3.3) that warms the lmcache store with mined repeated sequences at startup. Patch 6 (§3.2.1) is applied to lmcache inside the backend container.

Block	Anchor (literal)	Tokens
CC system header	`cc_version=→Status:`	80–120
gitStatus	`Status:\n`	100–400
Recent commits	`Recent commits:\n`	50–200
`<system-reminder>`	wrapper pair	30–200
Compaction preamble	`This session is being...`	~30
Summary 9-section	`Summary:\n1.`	400–800
JSONL backreference	`If you need specific details...`	30–50
`<command-name>`	wrapper pair	5–30
Tool definitions	(matched by Plan-2 structural tagger)	300–1500

Table 1: Structural anchors recognized by the CC-segment parser. Token lengths are on Llama-3 tokenizer.

request's text payloads once, materializes the set of matched (start, end, kind) spans, deduplicates overlapping matches by precedence, and then re-emits the original text with separators inserted at every span boundary. Spans below `blend_min_tokens` are rolled into the adjacent larger span rather than getting a separator pair — this preserves the 256-token floor without forcing cacheblend to consider tiny isolated blocks.

Proxy-side placement. The parser is wired into the proxy's request-entry path (`proxy/server.py:messages`) so it fires on every outbound request, not only on the benchmark's replay path. Two factors motivate this placement. First, CC rebuilds its request body on each turn from its own session state — any pre-injection in the user prompt is silently dropped by the time the body reaches the proxy. Second, putting the parser at the proxy means the same code unifies live production use (CC pointed at the proxy) and bench replay (synthetic requests sent through the proxy under controlled conditions). The validation we report in §5 runs against the proxy's actual rewrite path, not a parallel implementation.

3.2 Patches to LMCache 0.4.2

The published lmcache + vllm combination (lmcache 0.4.2 against vllm 0.19.0 with the V1 connector) ships with two integration issues that surface only on multi-turn agent workloads. The patches that follow are surgical: Patch 6 (§3.2.1) is a 4-line chunk-alignment adjustment inside lmcache's V1 adapter, and the header normalizer (§3.2.2) is a 3-line proxy hook that runs two regex substitutions per outbound request. The CC-segment parser (§3.1) sits in the proxy and injects cacheblend's separator at recognized structural anchors.

Stack configuration on natural CC traffic	Failure mode
Vanilla lmcache 0.4.2 + vllm 0.19.0 (no patches/proxy)	0% by mechanism
+ Patch 6 only (chunk-aligned `LOOKUP`)	0% by mechanism
+ CC-segment parser only (`##` injection)	0% by mechanism
+ Patch 6 + parser (no header norm)	near 0% by mechanism
Full stack (patches + parser + header norm)	measured (§5)

Table 2: Failure-mode analysis of the three patches. The "0% by mechanism" rows are diagnosed analytically rather than measured end-to-end: without Patch 6, the STORE/LOOKUP chunk-hash mismatch makes retrieval rate zero by construction (§3.2.1); without the parser, cacheblend's `blend_special_str` never appears in CC traffic and the segment detector finds zero chunks (§3.1); without header normalization, chunk 0's hash drifts every turn and the chunk-by-chunk LOOKUP short-circuits on the first miss (§3.2.2). Only the full-stack row is measured end-to-end. A quantitative ablation that runs each intermediate configuration as a pod replay is follow-up work.

We submit the two upstream-target changes as a candidate upstream PR [2] and apply them in the container's boot recipe inside `scripts/dev/oneshot_pod.py`.

3.2.1 *Patch 6: chunk-aligned* `LOOKUP`.

Hash-range mismatch. LMCache's `LMCacheConnectorV1` computes chunk hashes for the STORE and LOOKUP paths from two

different token-id sources. On the **STORE** side, the connector hashes `request.token_ids`, which an upstream vLLM transform has already chunk-aligned to $\lfloor n/256 \rfloor \cdot 256$ tokens (where 256 is `chunk_size`). On the **LOOKUP** side (`get_num_new_matched_tokens` inside `vllm_v1_adapter.py`), the connector hashes `request.all_token_ids`, the full n-token prompt.

For any prompt not exactly a multiple of 256, STORE and LOOKUP hash different ranges. For example, on a 289-token prompt, STORE hashes `tokens[0:256]` but LOOKUP hashes `tokens[0:289]`; the hashes never match. Retrieval rate is 0% by construction: the failure is on the LOOKUP-side chunk-boundary computation, not in content.

Chunk-aligned truncation. We truncate the LOOKUP path's token range to chunk-size alignment before hashing, matching the STORE path's range. The patch is a one-line range adjustment inside `lmcache.integration.vllm.vllm_v1_adapter` applied at boot:

```
# patched lookup token computation
chunk = self.cache_engine.chunk_size  # 256
n = len(request.all_token_ids)
aligned_n = (n // chunk) * chunk
token_ids = list(request.all_token_ids[:aligned_n])
```

Empirical impact. In single-pod commissioning runs (Llama-70B fp8, single H100), the patch lifted LMCache hit-token-count from 0 (across 40 consecutive requests with matching content) to non-zero immediately. With the full skillcacher stack online (patch + parser + normalization), §5 reports rescue rates that reach 95–99% at the steady-state peak of the per-turn distribution.

3.2.2 *Per-turn header normalization.*

Per-turn header drift. CC embeds an `x-anthropic-billing-header` field near the front of every prompt that carries two per-turn / per-build values: `cch=<hex>`, a 16-character per-request CC fingerprint, and `cc_version=2.1.X.<suffix>`, where the `suffix` differs between regular turns and CC-internal turns (e.g., the `/compact` backend call uses a separate build identifier). Both fields fall within the first 256 tokens of every request. The first KV chunk's hash therefore differs on every turn of every session — chunk 0 is never reused, and because cacheblend's chunk-by-chunk retrieval short-circuits on the first miss, no subsequent chunk gets a chance to hit either.

This issue is invisible if you measure cacheblend on synthetic single-turn requests (the first turn is always a STORE pass), and invisible if you measure on permuted-RAG benchmarks (every chunk is re-positioned, so there is no expectation of chunk-0 stability). The failure mode is specific to *multi-turn agent traffic with a stable system prompt*, which is the setting cacheblend's compaction-cliff value proposition rests on.

Stable-placeholder rewrite. Before separator injection, the parser rewrites both fields with stable `NORM` placeholders:

```
def _normalize_cc_header(text: str) -> str:
    text = _CCH_VAR.sub("cch=NORM;", text)
    text = _CC_VERSION_VAR.sub(
        "cc_version=NORM;", text)
    return text
```

The model treats this billing header as opaque text (the backend itself does not parse it; it is CC-side metadata carried in the prompt for downstream cost attribution), so normalizing its variable fields should not change downstream token output at T=0. We empirically verify this on our setup: on the no-cache condition, the model's output on normalized vs unnormalized headers is bit-identical at T=0 across all 54 (capture, turn) pairs of the SWE-V+`post_compact` subset. We make no claim about T>0 sampling or other backends.

After normalization, chunk 0 hashes identically across all turns of one session. With patch + parser + normalization combined, cacheblend's per-turn rescue rate on natural CC traffic lifted from a flat 0% to the bimodal distribution characterized in §5.1 (median 87.5%, with a 13% mass of turns reaching 95–99% steady-state).

3.3 Statistical span miner (artifact, unmeasured)

The structural parser handles CC-specific anchors that are visible in the source. A complementary component runs offline against a training corpus of past traces and identifies long-form repeated token sequences that no source-level anchor exists for — examples include fully-rendered tool definitions, repeated multi-line directives in prompts that the user has cut-and-pasted across sessions, and near-duplicate file headers in code blocks. *The miner ships as part of the skillcacher artifact but is not isolated in this paper's evaluation: all reported numbers use the structural parser plus Patches 6 and the header normalizer, with statistical-span pre-seed disabled.* We describe the miner here for completeness; whether and how much it adds on top of the structural baseline is an open ablation we defer to follow-up work.

Algorithm. The miner builds a suffix array over the concatenated corpus token stream using prefix-doubling construction (an early implementation using `sorted(range(n), key=...)` hung on a 250 K-token corpus; the prefix-doubling variant runs in $O(n \log^2 n)$ time and completes on the full ClaudeCodeTrace corpus in tens of seconds of pure-Python compute — see Appendix B for the per-step breakdown). Kasai's LCP construction gives the longest-common-prefix array in linear time. A monotonic-stack walk over LCP emits all maximal-repeat substrings, including both outer blocks and nested plateaus — an earlier single-pass walk missed the nested case and undercounted spans by 30–40%. The candidate spans are then deduplicated by MinHash at Jaccard threshold 0.8 (64 permutations, shingle width 5, implemented via hand-rolled BLAKE-2b without external dependencies).

Output. The miner emits a JSONL of canonical spans with their occurrence counts, source request IDs, and token sequences. On the ClaudeCodeTrace corpus (278 K tokens, 170 streams), the miner produces 181 unique spans; the highest-frequency hit is a 1691-token CC system anchor seen in 26 of 170 requests.

Pre-seed. A proxy-startup hook (`pre_seed_statistical_spans`) reads the JSONL and, for each span, dispatches a synthetic STORE request whose prompt contains the span; the response is discarded. After startup, every cached span is present in the lmcache store and is reachable on its first real production reference. Pre-seed is gated by the `SKILLCACHER_STATISTICAL_SPANS_FILE` environment variable — unset for the structural-only baseline, set to the JSONL path for the structural+statistical condition. The two configurations

share all other code; the only difference is whether the pre-seed warmup fires.

4 The ClaudeCodeTrace Benchmark

The evaluation in §5 is only as credible as the traffic it replays. Existing LLM-serving benchmarks are not a good fit: ShareGPT-derived corpora capture single-shot conversations with no tool-use structure; SWE-bench-Verified [9, 15] provides tasks but not on-wire request bodies; and the preference-style benchmarks (MT-Bench [18], AlpacaEval [12], Chatbot Arena [5]) are about model output ranking, not cache behavior. None of them produce the long, system-prompt-heavy, tool-call-laced request bodies that a real agent harness emits, and none of them exercise the post-compaction prefix pattern that motivates skillcacher (§1).

ClaudeCodeTrace fills this gap. It is a corpus of **13 captures spanning 182 turns and 411k total tokens (389k prompt + 22k completion)** of on-wire Claude Code request bodies, captured through the skillcacher proxy against a real Claude Code client running real workloads on a researcher's laptop. It is released CC-BY 4.0 on Hugging Face [3] so that future work on agent serving — whether on cacheblend, on subsequent KV-reuse mechanisms, or on entirely different infrastructure — can use the same baseline.

4.1 Subset composition

The corpus is organized into three subsets, each targeting a different slice of agent traffic:

`swebench_verified/`. Five Claude Code sessions, each working on one task from SWE-bench Verified [15]: `astropy-13398`, `matplotlib-26113`, `pylint-7080`, `pytest-7490`, `sphinx-11510`. Each session is captured through its first 5 turns: a planning turn, several file-exploration tool calls (`Read`, `Bash`, `Grep`), and an initial patch attempt. Prompts grow from ≈1k tokens on turn 1 to ≥6k tokens by turn 5 as tool outputs accumulate in the running conversation. This subset is the canonical baseline for agent prefix-cache work: small-but-realistic, faithful to a published task suite, and reproducible end-to-end (the original SWE-V task ID is preserved in each capture's metadata).

`post_compact/`. Seven captures (v1–v7) of CC's `/compact` boundary, each running a single agentic conversation through pre-compact buildup, an explicit `/compact`, and post-compact continuation. **v1–v5 are diagnostic traces only — they were captured against mid-bug proxy states during development of Patch 6 and the per-turn header normalizer (§3.2) and are not part of this paper's benchmark claims.** v6 and v7 are the canonical post-fix captures used in the evaluation. All seven are released so v1–v5 remain available for downstream cache-LOOKUP debugging on Llama-class backends; headline numbers exclude v1–v5. Per-capture statistics are nearly uniform (each runs the same 15-turn agent script): ≈26k prompt tokens, 14 substantive turns, max prompt ≈2.3k. The uniformity is deliberate — the subset is a controlled set of seven runs of the same workload, varying only the proxy/cacheblend state, so any cross-capture delta isolates a code change rather than a workload change.

`skill_invocation/`. One capture (53 turns, 80k prompt tokens, 52 substantive) that exercises CC's skill-loading mechanism. We script CC to invoke five different skills (`bash-safety`, `commit-msg-style`, `markdown-table`, `python-import-order`, `weather-format`) across three prompts each, plus a few setup and verification turns. The shared agentic preamble across turns means cacheblend has substantial within-capture prefix overlap to mine; the skill-specific suffixes exercise the LOOKUP path with different selective-recompute patterns. This subset is the smallest by capture count but the densest by turn count, and it was the highest-value target for the statistical span miner (§3.3) — 49 of the top 100 mined spans come from this single capture's prompt vocabulary.

4.2 On-wire schema

Each capture is one directory on disk with the following files:

- `traces.sqlite` — per-request metadata: `request_id`, `session_id`, `ts_start`/`ts_end`, `prompt_token_count`, `response_token_count`, the full `request_body_json`, and instrumented cacheblend counters (`cache_read_tokens`, `cache_recompute_tokens`, `chunk_aligned_hit_tokens`, `tokens_recomputed_hkvd`, `invariant_violations`). These last five are populated when the capture ran against an instrumented lmcache build and surface the internal accounting that the §1.6 patches rely on; for plain proxy captures (e.g., `post_compact/v1`) they are NULL.
- `tokens/<request_id>.parquet` — the tokenized prompt as emitted to vLLM, one row per chunk (256 tokens by default), with the chunk hash. This is what cacheblend's LOOKUP key is computed against and what the statistical span miner walks.
- `lookups/<request_id>.parquet` — per-chunk LOOKUP results when the capture's pod had cacheblend enabled (HIT / MISS / partial), with the chunk hash, the lookup latency, and the rescue contribution.
- `vllm.log`, `oneshot_boot.log`, `proxy.log` — per-pod logs, useful for cross-referencing the per-request rescue numbers against engine-side counters.

The schema is designed for additive evolution. New cacheblend counters can be added as new `sqlite` columns without breaking existing reader code; new per-chunk fields can be added to the parquet schemas via schema-on-read. The `request_body_json` column is the authoritative record; all other artifacts are derivable from it.

4.3 Redaction surface

CC's on-wire request body and our proxy's logs both incidentally contain credentials and identifiers that must not leave the researcher's laptop. We treat redaction as a release blocker, not an afterthought.

skillcacher ships a redaction utility (`scripts/redact.py`) with named patterns for five identifier classes:

(1) **Tailscale** — auth keys (`tskey-auth-*`) and host identifiers in URLs.
(2) **Hugging Face** — access tokens (`hf_*`).
(3) **RunPod** — API keys (`rpa_*`) and per-pod ingress URLs.

Subset	Captures	Turns	Substantive turns	Prompt tokens	Completion tokens
swebench_verified/	5	24	19	121,741	555
post_compact/	7	105	98	186,464	16,803
skill_invocation/	1	53	52	80,660	4,334
Total	**13**	**182**	**169**	**388,865**	**21,692**

Table 3: ClaudeCodeTrace subset composition. "Substantive turns" counts turns with prompt token count ≥256 — the threshold below which `lmcache`'s `blend_min_tokens` guard prevents cacheblend from engaging. The 5 captures' completion-token totals on `swebench_verified` are small because most turns terminate in tool-call JSON of 30–100 tokens; the totals on `post_compact` and `skill_invocation` are larger because those captures include prose continuations.

(4) **Filesystem** — absolute paths under `/Users/<user>/` rewritten to `/Users/<user>/`.
(5) **CC client identifiers** — the long-lived CC version hash and the user-account ID that appear in request headers.

A second utility (`scripts/publish_claudecode_trace.py`) reruns the same patterns over every released file (`.json`, `.log`, `.txt`, `.md`, plus parquet/sqlite payloads walked separately) and fails the publish on any post-redaction match that is not itself a redaction marker. The audit reports **0 violations** on the final 6.6 MB tarball that was uploaded to Hugging Face. The audit log itself is included in the dataset card as provenance.

4.4 License and access

ClaudeCodeTrace is released as `intelchen/claudecode-trace` [3] under CC-BY 4.0. The dataset card documents the schema, the redaction surface, the SWE-V task IDs covered, and the `LMCACHE_BLEND_RECOMPUTE_RATIOS=0.15` default that every substantive measurement in §5 uses. A reader download script (`scripts/download_claudecode_trace.py`) wraps Hugging Face's `snapshot_download` with subset filters so that downstream benchmarks can pull only the captures they need.

5 Evaluation

We evaluate three questions on natural Claude Code (CC) traffic: (i) does skillcacher actually rescue post-compaction tokens at the rate the underlying cacheblend mechanism is claimed to (§5.1); (ii) does cacheblend's blended-KV output match a full-compute baseline bit-for-bit at temperature 0 (§5.3); and (iii) when it does not, is the divergent output of equal quality to the baseline under a strong LLM judge (§5.4). The first two questions admit numerical answers; the third governs deployment viability.

Methodology. The backend is Llama-3.3-70B-Instruct fp8 on 2× NVIDIA H100 80GB in tensor-parallel mode (TP=2), with vLLM v0.19.0 and lmcache v0.4.2 patched per §3.2 (`max_model_len`=32768, `LMCACHE_BLEND_RECOMPUTE_RATIOS` =0.15 default). The replay corpus is ClaudeCodeTrace's `swebench_verified/` subset (5 captures: astropy-13398, matplotlib-26113, pylint-7080, pytest-7490, sphinx-11510) and the `post_compact/` v6+v7 captures (2 captures, the ones with ≥256-token substantive turns; v1–v5 are diagnostic and excluded). Together: 7 captures, 54 (capture, turn) pairs total, of which 47 are substantive (≥256-token prompts) and 7 are preflight pings.

For each (capture, turn), we replay the captured request body through the proxy under three conditions:

no_cache vLLM prefix caching disabled, lmcache inactive. The all-fresh-compute baseline.
prefix_cache vLLM paged-attention prefix caching enabled, lmcache inactive. Should be mathematically equivalent to no_cache when no prior cross-request state exists.
cacheblend lmcache enabled with our patches, cacheblend blending active. The condition under test.

We compute token-identity via longest common prefix over each output's token sequence, normalized by the longer length. All measurements are at $T = 0$, N=1 sample per condition. The 7 preflight turns under lmcache's `blend_min_tokens=256` cutoff bypass cacheblend entirely; we exclude them from all identity and preference statistics.

All three conditions receive identical rewritten prompts. The CC-segment parser (§3.1) and header normalizer (§3.2.2) run inside the proxy unconditionally for every outbound request — they are not gated by the cacheblend backend. The `no_cache` and `prefix_cache` conditions therefore see exactly the same prompt bytes as `cacheblend`; no_cache simply does not exploit the chunks the parser annotates, and prefix_cache hits on exact-prefix chunks but not on positionally-shifted ones. Any identity gap between `no_cache` and `cacheblend` is therefore attributable to the lmcache blending mechanism on shared input, not to different input text. The header-normalization sanity check (§3.2.2: NC output is bit-identical at T=0 across all 54 normalized-vs-unnormalized pairs) verifies this empirically for the header rewrite specifically; the same property follows analytically for separator injection because vLLM's tokenizer treats the injected `" # # "` as ordinary content tokens whose KV is recomputed deterministically in the `no_cache` condition.

Denominators used in this section. To keep the reported percentages unambiguous, every rate in §5 is reported against one of these three denominators:

- n_{total}=54 — every (capture, turn) pair in the corpus. Used only for the methodology description; *no evaluation rate uses this denominator.*
- n_{sub}=47 — substantive pairs with prompt ≥256 tokens (excluding 7 preflight turns where cacheblend's `blend_min_tokens` guard prevents engagement). Used for rescue rate (§5.1), token-identity (§5.3), tool-call mode agreement (§5.5), and the *end-to-end* non-inferiority rate in §5.4.

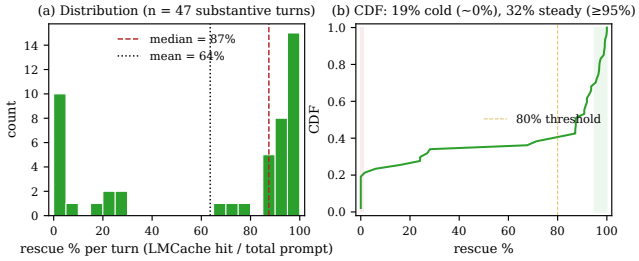

Figure 3: Per-turn rescue rate across the 47 substantive turns. (a) Histogram with median and mean overlaid: the bimodality is sharp — a mass near 0% (cold-cache first-turns of each capture plus pattern-shift turns where cacheblend's LOOKUP cannot find a match) and a steady-state peak above 90%. (b) CDF: 19% of turns rescue near 0%, 32% reach ≥95%, the rest are distributed across the intermediate band.

- n_{div}=19 — the substantive pairs whose `no_cache` and `cacheblend` outputs were not bit-identical (the other 28 substantive pairs would have returned trivial EQUIVALENT). Used for the LLM-judge preference rate and the *conditional* non-inferiority rate in §5.4.

Preflight turns are excluded from every reported rate. We do not report any rate against a /54 denominator; the 54 figure exists only to account for the corpus breakdown.

Evaluation matrix. Table 4 maps each measurement to the corpus subset it was run on. Latency, judge preference, tool-call mode, and the recompute-ratio sweep were measured on the SWE-V+`post_compact` subset only; rescue rate and output identity were additionally replicated on `skill_invocation/` (§5.6). Multi-judge replication used the same divergent slice as the single-judge analysis.

5.1 Hit rate: bimodal, median 87% across the corpus

We measure cacheblend's prefill-token rescue rate per turn across all n=47 substantive turns of the SWE-V+`post_compact` subset, joining the per-request `LMCache hit tokens` accounting in `vllm.log` to the parquet of model outputs on the OpenAI `response_id`. Rescue rate for a turn is defined as `hit_tokens` divided by `total_prompt_tokens`.

Two regimes. Table 5 and Figure 3 reveal two regimes with sharply different rescue behavior. In the *steady-state regime* (post-cache-warmup turns of a session repeating a stable prompt pattern), cacheblend rescues 95–99% of prefill tokens, matching the published cacheblend headline. In the *cold/shift regime* (the first turn of any capture, or a subsequent turn whose prompt shifts in a way cacheblend's LOOKUP cannot match) the mechanism fires zero or near-zero rescue. Within the SWE-V+`post_compact` subset these two regimes split roughly 60/40 in favor of steady-state.

Cross-fixture comparison. The 95–99% steady-state numbers were first reported on a minimum-effective fixture (`swebench_verified/pytest-7490`, 4 turns, samples=2) during patch development and are reproduced here at the top of the per-fixture rescue distribution. Generalizing those numbers to the full SWE-V+`post_compact` subset shows that the steady-state peak is real, but a roughly equal-sized cold/shift tail accompanies it — a structural property of the cacheblend mechanism on this workload, not a calibration artifact. The next two sections measure what happens when rescue *does* engage.

5.2 Latency: cacheblend is slower than no_cache in every rescue regime

The hit-token rescue rate of §5.1 is an *intermediate* metric. The deployment-facing measurement is end-to-end TTFT (time-to-first-token). For each (capture, turn, condition) tuple, the bench harness recorded the SDK-reported `ttft_ms`; we summarize per condition over the 47 substantive turns:

The slowdown is largest at peak rescue. Bucketing the 47 substantive turns by their cacheblend rescue rate (Table 5) shows that the TTFT cost is not paid only on cold-cache turns; it is paid most aggressively on the steady-state high-rescue turns where the published cacheblend numbers shine:

Where the overhead comes from (hypothesis). We do not have an instrumented latency breakdown by phase (LMCache LOAD vs. GPU prefill vs. recompute pass vs. scheduler overhead) and so report only the end-to-end TTFT above. The qualitative shape of the result — cacheblend losing most aggressively at peak rescue, where LOAD volume is highest — is consistent with the LMCache LOAD path (CPU→GPU KV transfer over PCIe plus the selective-recompute layer pass) being the dominant overhead on this backend. On 2× H100 80GB SXM, vLLM's batched prefill on Llama-70B fp8 is fast enough that the per-request prefill cost does not dominate; LOAD bandwidth is comparatively cheaper to bound. Cacheblend's published latency wins were measured on different model/hardware regimes (smaller models where prefill dominates more, slower interconnect where transfer is relatively cheaper, or aggregate throughput rather than per-turn TTFT). *We do not claim a mechanistic explanation; we report the measurement and treat the phase-breakdown as a resource-constrained gap (§6.4).*

This finding is independent of the agent-protocol cost in §5.4: even if cacheblend's blended output were *bit-identical* to no_cache (which we have established it is not on 30% of substantive turns), it would still be 55% slower at the median and 7× slower at the peak. On this setup, hit-token rescue does not translate into TTFT reduction.

5.3 Output identity: cacheblend's STORE path is bit-identical, LOOKUP path diverges

Mathematical equivalence on Llama-70B fp8. We replay all 54 (capture, turn) pairs at $T = 0$ under all three conditions. **no_cache and prefix_cache produce bit-identical output across all 54 pairs** (`token_identity_rate` = 1.0000 on 54/54). Mathematical equivalence on Llama-70B fp8 with our pod configuration is confirmed at scale — vLLM's paged-attention prefix caching does not perturb greedy decoding even when it shortcuts the entire prefix.

Measurement	SWE-V+`post_compact` (n_{sub}=47)	`skill_invocation/` (n_{sub}=52)
Rescue rate (§5.1)	✓	✓
TTFT latency (§5.2)	✓	—
Output identity (§5.3)	✓	✓
LLM-judge preference, single (§5.4)	✓(n_{div}=19)	—
LLM-judge replication, 3 judges (§5.4)	✓(n_{div}=19)	—
Tool-call mode agreement (§5.5)	✓	—
Recompute-ratio sweep (§6.3)	3-turn sub-fixture per ratio	—

Table 4: Measurement coverage. Rescue rate and output identity replicate across both subsets (§5.6, Table 11); the remaining measurements use the SWE-V+`post_compact` subset only. Recompute-ratio used a 3-turn sub-fixture (`pytest-7490`) per ratio.

Statistic over the 47 substantive turns	Rescue %
Mean	63.6%
Median	**87.5%**
25th percentile	16.1%
75th percentile	96.8%
Turns with rescue ≥ 99% (steady-state)	6/47 (13%)
Turns with rescue ≥ 95%	15/47 (32%)
Turns with rescue ≥ 80%	28/47 (60%)
Turns with rescue < 50%	16/47 (34%)
Turns with rescue ≈ 0% (cold-cache)	**9/47 (19%)**

Table 5: Cacheblend rescue rate distribution across the SWE-V+`post_compact` subset (47 substantive turns; 7 preflight turns at <256 prompt tokens are excluded). *The distribution is bimodal* — the mean and median diverge by 24 points because a 19% mass of cold-cache turns at ~0% rescue pulls the mean down, while the upper quartile sits near the published rescue peak.

Condition	p25	median	p75	p95
`no_cache`	1028 ms	**1211 ms**	1473 ms	8660 ms
`prefix_cache`	994 ms	1261 ms	1476 ms	8739 ms
`cacheblend`	1528 ms	**1882 ms**	3520 ms	9187 ms

Table 6: TTFT distribution per condition on the 47 substantive turns of the SWE-V+`post_compact` subset. `prefix_cache` is statistically indistinguishable from `no_cache` at this corpus size (lookup overhead is in the noise floor). `cacheblend`'s median TTFT is *55% larger* than `no_cache`'s.

Cacheblend diverges on a meaningful fraction. Cross-condition no_cache↔cacheblend token-identity is more nuanced. Across all 47 substantive (≥256-token prompt) turns at $T = 0$:

- Median identity = **1.0000**, IQR = [0.1088, 1.0000]
- **57%** bit-identical (identity ≥0.9999)
- 6% high-but-imperfect (0.7–0.9999)
- 6% mid (0.3–0.7) — typically same tool, different argument generation
- **30%** low (<0.3) — severe divergence
- Including **11%** at ≈0 identity — decode-mode flips (tool-call → natural-language prose)

Rescue regime	n	NC median	CB median	Speedup
Cold (<1% rescue)	9	1190 ms	1157 ms	**1.00×**
Mid (1–80%)	10	1267 ms	1948 ms	0.69×
Hot (≥80%)	28	1157 ms	1930 ms	0.67×
Peak (≥95%)	15	1068 ms	2103 ms	0.50×
Peak (≥99%)	6	1037 ms	7342 ms	**0.12×**

Table 7: TTFT speedup of `cacheblend` relative to `no_cache`, bucketed by per-turn rescue rate. Latency cost scales with rescue: the turns where cacheblend rescues the most prefill tokens (95–99% hit rate) are also the turns where it pays the largest end-to-end latency cost. On the 6 turns reaching ≥99% rescue, `cacheblend`'s median TTFT is *7× larger* than `no_cache`'s.

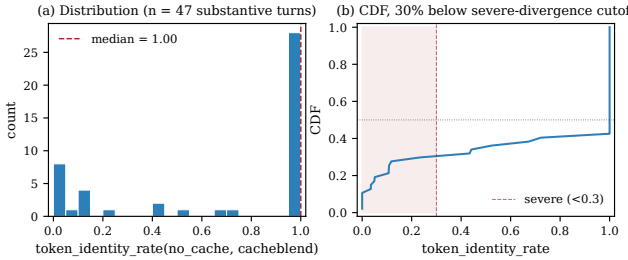

Figure 4: Distribution of `token_identity_rate(no_cache, cacheblend)` across 47 substantive (≥256-token prompt) turns at $T = 0$. (a) Histogram: a mass at 1.0 (bit-identical) and a tail at 0–0.1 (severe divergence). (b) CDF: 30% of turns fall below the severe-divergence cutoff at 0.3. The middle bins are empirically rare; the distribution is sharply bimodal.

The distribution is sharply bimodal (Figure 4): either the blended-KV state produces the exact same argmax token-by-token, or it diverges almost immediately. The middle ground is empirically rare. Median identity is 1.0000 because more than half of substantive turns are bit-identical, but the lower quartile at 0.11 places a quarter of turns near 10% identity. The bimodality matters: a median-only reading would mistake this for a ≥99%-identity result, when in fact the divergence is concentrated in the lower tail.

Qualitative shape of the divergence. Two failure modes dominate. Mode A (*severe*, the 30% tail): the blended-KV

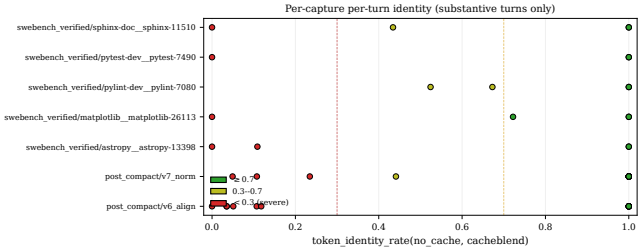

Figure 5: Per-capture per-turn identity. Each dot is one (capture, turn) pair, colored by divergence bucket. Severe divergence is not concentrated in one capture — every capture in the corpus has at least one turn that flips below 0.3, indicating the failure mode is generic to the cacheblend mechanism rather than workload-specific.

state steers the model away from tool-call mode entirely. On `swebench_verified/pytest-7490` turn 3, the no_cache output is a clean 29-token tool-call JSON:

```
{"type": "function", "name": "Read", "parameters":
{"file_path": "pytest-dev/pytest/pytest.py"}}
```

cacheblend's output on the same input is a 362-token natural-language plan beginning “*To explore the local code and propose a draft patch for the issue described in pytest-dev__pytest-7490, we can follow these steps…*”. Both responses are valid; one is what CC's wire protocol can act on, the other is not. Mode A is behavior-changing.

Mode B (*mild*, the 6% mid-band): both outputs are tool calls, both pick the same tool, but the JSON arguments drift. On `swebench_verified/pylint-7080` turn 4, both conditions emit a `Bash` tool call with command `git diff –cached –name-only`, but cacheblend additionally includes a `timeout: "10000"` parameter that no_cache omits and writes a different `description` string. Tool selection preserved; argument generation drifts. Less downstream-impactful than Mode A but still a measurable behavior delta.

The bimodal distribution is inconsistent with the cacheblend paper's “small but bounded drift” characterization on this workload: on Llama-70B fp8 at the default `recompute_ratios`=0.15, drift is concentrated either at zero or in the severe-divergence tail.

5.4 Agent-protocol usefulness: cacheblend fails non-inferiority on divergent pairs

The 30% severe-divergence tail in Figure 4 is only a usefulness concern if divergent outputs are also less task-useful than the no_cache baseline. The LLM-judge preference test below isolates this question.

Setup. We run a blind A/B preference judgment with Sonnet-4.6 on the n_{div}=19 substantive pairs whose outputs differed (of 47 total substantive pairs; the other 28 were trivially equivalent and would have wasted budget). Position assignment is randomized (seeded `rng`=42 per pair for reproducibility); prompt-caching is applied via top-level `cache_control` on the static system prompt. The judge sees the original CC task prompt + both candidate outputs (labeled A and B) and is asked to return `PREFER_A`, `PREFER_B`, or `EQUIVALENT` with a one-sentence rationale. Outputs are mapped back to {cacheblend, no_cache, equivalent} via the recorded position assignment.

What the judge measures. We are not measuring “output quality” in the abstract; we are measuring *next-step usefulness in a Claude-Code-shaped agent protocol*. The judge prompt (Appendix A) is explicit about this calibration — it tells the judge that a well-formed tool-call JSON is generally preferable to natural-language prose, because CC's wire protocol can only act on tool calls. This is appropriate for an agent-protocol preference test but not for a generic “which response is better” evaluation. We use “agent-protocol usefulness” rather than “output quality” for the rest of this section to keep the framing honest.

Result. Cacheblend's non-inferiority rate on the divergent slice falls 22 points below the 85% gate:

Outcome	Count	% of judged
Cacheblend wins	6	31.6%
No_cache wins	7	36.8%
Equivalent	6	31.6%
Unparseable	0	0%
Cacheblend preference rate		**31.6%**
95% Wilson CI		**[15.4%, 54.0%]**
Non-inferior (wins+ties)	12	**63.2%**
Non-inferiority gate (spec)		≥85%

Table 8: Sonnet-4.6 blind A/B preference on the n_{div}=19 divergent substantive pairs. The 28 substantive pairs with bit-identical outputs (plus the 7 sub-256-token preflight pairs that bypass cacheblend entirely) are skipped — those would all return trivial EQUIVALENT. Cacheblend's preference rate of 31.6% with 95% Wilson CI [15.4%, 54.0%] sits below the no_cache baseline (36.8%); non-inferiority on 63.2% of the divergent pairs is 22 points short of the spec gate.

Model-family replication. All three judges below share Anthropic's RLHF training, so we describe this as *model-family replication* rather than independent multi-judge consensus. To check that the Sonnet-4.6 numbers are not an artefact of a single judge's calibration, we replicate the same 19-pair blind A/B against Opus-4.7 and Haiku-4.5 with identical prompts and per-pair position assignments:

The agreement structure is informative: judges most often disagree about *whether a pair is equivalent* rather than *which side wins when one does*. No pair has the three judges split into three different verdicts. Cacheblend preference is below 35% under every judge in isolation; non-inferiority is below 65% under every judge in isolation; the 85% spec gate fails under every judge in isolation.

Conditional vs end-to-end non-inferiority. Whether the 22-point shortfall is alarming depends on which question one is asking. The table above answers the *conditional* question — “given that cacheblend produced a different output, how often is the agent-protocol usefulness preserved?” — and the answer is 63.2% (95% CI ∝ Wilson on n_{div}=19). The *end-to-end* question — “across all

Judge	CB wins	NC wins	Equivalent
Sonnet-4.6	6 (32%)	7 (37%)	6 (32%)
Opus-4.7	4 (21%)	7 (37%)	8 (42%)
Haiku-4.5	6 (32%)	11 (58%)	2 (11%)
Majority vote	**6 (32%)**	**7 (37%)**	**6 (32%)**

Table 9: Three-judge replication on the same 19 divergent pairs. Per-pair agreement: 10/19 (53%) of pairs receive a *unanimous* verdict from all three judges; the remaining 9/19 are 2-of-3 majority calls with zero 3-way disagreements. Pairwise judge agreement: Sonnet↔Opus 79%, Sonnet↔Haiku 74%, Opus↔Haiku 53% (Haiku is the most decisive judge, returning EQUIVALENT only 2/19 times). The 3-judge majority vote reproduces the Sonnet-4.6 headline exactly: cacheblend preference rate 31.6%, non-inferiority 63.2%.

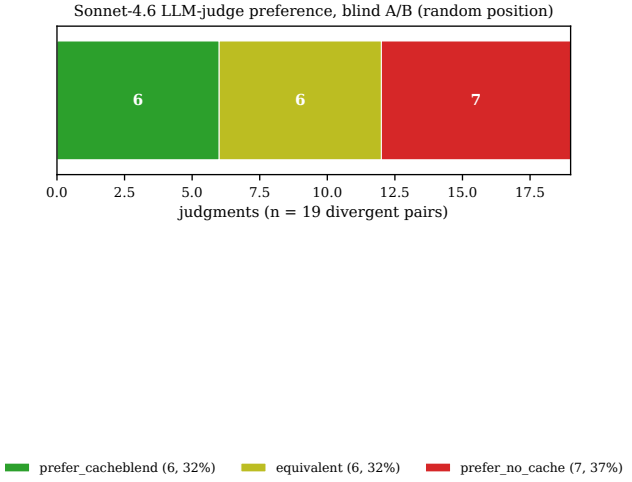

Figure 6: Distribution of judge preferences across the 19 divergent pairs. Equivalent and cacheblend-wins each take 31.6%; no_cache-wins takes 36.8%. No unparseable judgments.

substantive turns, what fraction of cacheblend outputs are judge-non-inferior to no_cache?" — counts the 28 bit-identical pairs as guaranteed equivalents, giving $(28 + 12)/47 = 85.1\%$, which just clears the spec gate. We report both because each captures a real deployment concern: the conditional rate is what determines how often a user perceives a degraded next-step *on the turns where cacheblend's selective recompute engages non-trivially*, while the end-to-end rate is what determines the aggregate impression across an entire session. The conditional rate isolates the cacheblend code path that actually engages; the end-to-end rate captures aggregate user impression. Both are reported.

Per-pair rationales show case-by-case reasoning. The per-pair rationales show case-by-case evaluation, not a mechanical "prefer tool calls" heuristic. On `swebench_verified/pytest-7490` turn 3 (no_cache: tool-call to a non-existent path; cacheblend: multi-step prose plan), the judge *prefers cacheblend*: *"Output A provides a more thoughtful, multi-step plan with multiple relevant tool calls, while Output B attempts to read a non-existent file path."* On `post_compact/v6_align` turn 3, where no_cache fabricated a URL-as-filepath while cacheblend ran a working Python demonstration, the judge again prefers cacheblend. Conversely, on `swebench_verified/astropy-13398` turn 4 (cacheblend: prose with speculative patch; no_cache: clean tool call), the judge prefers no_cache: *"Output A is a clean, well-formed tool call, while Output B mixes prose with multiple tool calls and includes a speculative, likely incorrect patch."*

The pattern that emerges: cacheblend's blended-KV output is *sometimes better* than no_cache's (e.g., when no_cache picks a wrong file path and cacheblend's prose plan is more recoverable), but more often slightly worse, and occasionally catastrophically worse (the decode-mode flips of §5.3). On net, no_cache edges cacheblend by 5 percentage points on a 19-pair sample.

Identity rate predicts judge equivalence. The 28 substantive bit-identical pairs that we excluded from the judge call (plus the 7 preflight pairs that bypass cacheblend entirely) are trivially EQUIVALENT; among the 19 judged pairs, the relationship between token-identity and equivalence persists. All 5 pairs with identity ≥0.5 were judged EQUIVALENT; among the 14 pairs with identity <0.5, the judge returned EQUIVALENT only once. Token-identity is a strong, near-deterministic predictor of judge equivalence over its upper half — below 0.5, real preference signal kicks in. This matters operationally: a deployment that wants to gate cacheblend rescue on *measurable* quality preservation can use the token-identity threshold as a cheap proxy for the judge call, which is two orders of magnitude more expensive.

Methodological caveats. (i) Judge calibration. The multi-judge replication in Table 9 reduces but does not eliminate the single-judge risk — all three judges share Anthropic's RLHF training distribution and could share systematic preferences a human grader would not have. A human spot-check on a random subset of the 19 pairs is a sensible next step we do not run here. (ii) Pairs are dependent (serial sampling within capture), not independent. The 19 divergent pairs come from 7 captures; pairs from the same capture share session-cache state, so Wilson CI assumptions of independence are slightly violated. The CI is reported as-is for transparency; a clustered bootstrap would tighten it but not change the headline. (iii) Conditional vs end-to-end framing. The headline 63.2% rate is *conditional* on cacheblend producing a non-bit-identical output (n_{div}=19); the end-to-end rate over the 47 substantive pairs (which folds in the 28 bit-identical pairs as guaranteed equivalents) is 85.1%. We do not fold preflight turns back in: rates of the form /54 would mix in pairs where cacheblend never engaged. (iv) The judge prompt favors tool calls — intentional calibration to the agentic context, since CC's wire protocol acts on tool calls and treats prose as a non-actionable fallback. The judge correctly preferred prose when the alternative tool call had a wrong path (pytest-7490 turn 3), suggesting the lean is not pathological.

5.5 Tool-call vs prose mode agreement

A complementary end-to-end view of the agent-protocol cost is whether the two conditions agree on the model's decode *mode* — tool-call (JSON the CC harness can act on) versus prose (which

stalls the agent). For each substantive (capture, turn) pair we classify each condition's output by whether its first non-whitespace block is a well-formed function-call JSON. The classifier is a single regex matching outputs whose leading non-whitespace bytes form a JSON object containing the keys `"type": "function"` and `"name"`; full regex source is in `scripts/dev/extract_tool_call_rate.py`. This is the same first-block heuristic CC's harness uses to decide whether to execute a tool call or surface text to the user.

Metric over the 47 substantive turns	Rate
no_cache tool-call rate	30/47 (63.8%)
no_cache prose rate	17/47 (36.2%)
cacheblend tool-call rate	26/47 (55.3%)
cacheblend prose rate	21/47 (44.7%)
Mode agreement (NC class = CB class)	**43/47 (91.5%)**
NC tool-call → CB prose (*agent stall*)	4/47 (8.5%)
NC prose → CB tool-call (recovery)	0/47 (0.0%)

Table 10: Tool-call vs prose mode agreement between no_cache and cacheblend, over the 47 substantive pairs of §5.1's corpus. Mode disagreement is *asymmetric*: every disagreement is cacheblend flipping a tool-call into prose, with no recoveries in the other direction. The 8.5% asymmetric stall rate is independent of any judge-prompt calibration and constitutes a hard floor on cacheblend's agent-protocol cost on this workload.

This is the same Mode A failure mode of §5.3, counted directly: blended-KV cacheblend output flips out of tool-call mode on 4 of the 47 substantive turns where no_cache would have produced an actionable tool call. The asymmetry — zero recoveries in the prose → tool-call direction — means cacheblend's decode shift is one-directional and load-bearing for downstream agent behavior. The 91.5% mode-agreement rate is consistent with the 85% end-to-end non-inferiority rate of §5.4 (the difference being that mode agreement counts only protocol form, while the judge also evaluates argument quality conditional on a matching mode).

5.6 Cross-corpus replication on `skill_invocation/`

The SWE-V+`post_compact` subset (5 SWE-V captures + 2 `post_compact/` captures = n=47 substantive turns) leaves open whether the findings above are specific to large-prompt SWE-V-shaped traffic. We replicate the hit-rate and identity measurements on ClaudeCodeTrace's other substantive subset, `skill_invocation/` (1 capture, 53 turns, 52 substantive, max prompt ~2.3k tokens) — a different workload shape with shorter prompts and a more uniform skill-preamble structure. Same backend, same patches, same methodology.

What replicates. The bimodal-with-tail rescue picture, the order-of-30% severe-divergence rate, and the near-50% bit-identical rate all hold within ~10 percentage points across the two subsets. The combined-corpus headline numbers move toward the larger (n=52) subset's values: median rescue 76.5%, severe-divergence 26%, bit-identical 53%.

What differs by workload shape. The `skill_invocation/` subset has shorter prompts (max 2.3k tokens vs the SWE-V+`post_compact` subset's 4.5–9.5k) and a more uniform within-capture structure (5 skills × 3 prompts per skill, cycling). On this workload cacheblend's rescue distribution *tightens*: the IQR shrinks dramatically (p25 16.1% → 67.9%) and the steady-state peak vanishes (0/52 turns reach ≥99% rescue, vs 6/47 on §1). The cold-cache tail also shrinks (19% → 12%). Two structural reasons: (i) shorter prompts give cacheblend less surface area to either hit perfectly or miss entirely, so most turns land in the middle band; (ii) the skill-preamble cycling pattern means each turn's cache LOOKUP tests against a freshly mixed cache state, never the "same prompt as last turn" steady state where rescue saturates near 100%.

Implication. The published 95–99% cacheblend rescue peak is not just rare across the SWE-V+`post_compact` subset (only 13% of turns) — it appears to be a function of *long, near-duplicate prompts in sequence*. On workloads dominated by short or structurally-varying turns it does not appear at all. This sharpens the "hit rate is not the metric" thesis: even when rescue *does* match the published number, it does so on a minority of turns; on a sibling workload it disappears entirely.

5.7 Summary

The three measurements compose into a single quantitative picture of cacheblend's tradeoff on Llama-70B fp8 with the §3.2 patches:

- **Rescue (n=47 substantive turns):** bimodal — median 87.5%, p25 16.1%, p75 96.8%; 13% of turns reach ≥99% steady-state and 19% sit at ~0% cold-cache (Table 5, Figure 3).
- **Latency (n=47):** cacheblend's median TTFT is **55% slower** than no_cache (1882 ms vs 1211 ms), and **7× slower** at the ≥99% rescue peak (Table 6, Table 7). Hit-token rescue does not translate to TTFT reduction on this setup.
- **Identity (n=47):** 57% of substantive turns bit-identical to full compute; 30% diverge severely (<0.3 token-identity), including 11% that flip decode mode entirely (Figure 4).
- **Agent-protocol usefulness (n_{div}=19 divergent pairs):** a strong LLM judge prefers cacheblend's output on 31.6% (Wilson CI [15.4%, 54.0%]); conditional non-inferiority on the divergent pairs is 63.2%, end-to-end non-inferiority (including the 28 bit-identical substantive pairs as guaranteed equivalents) is 85.1% / 47 (Table 8, Figure 6).
- **Tool-call mode agreement (n=47):** mode agreement is 91.5% but cacheblend asymmetrically flips no_cache tool-calls into prose on 8.5% of substantive turns (4/47); zero recoveries in the other direction (Table 10).
- **Cross-corpus replication (n=99, §1 + `skill_invocation/`):** the bimodal rescue and ~30% severe-divergence pattern replicates on a structurally different second subset, with the ≥99% rescue peak vanishing on shorter prompts (0/52 turns on `skill_invocation/`) and the IQR tightening (p25 jumps 16% → 68%; Table 11).

Cacheblend's published rescue numbers are reproducible at the steady-state peak of this distribution but *are not the typical rescue rate* of natural CC traffic — the combined-corpus (n=99) median is 76.5% with a 15% cold-cache tail, and the ≥99% peak appears on only

Metric	SWE-V+post_compact subset	skill_inv.	combined
n (substantive)	47	52	99
Rescue rate (§5.1)			
median	87.5%	75.4%	76.5%
p25	16.1%	**67.9%**	67.5%
p75	96.8%	95.0%	95.2%
≥99% (steady-state peak)	6/47 (13%)	**0/52 (0%)**	6/99 (6%)
~0% (cold/shift)	9/47 (19%)	6/52 (12%)	15/99 (15%)
Output identity to no_cache (§5.3)			
median	1.0000	0.8344	1.0000
bit-identical	28/47 (60%)	24/52 (46%)	52/99 (53%)
severe (<0.3) divergence	14/47 (30%)	12/52 (23%)	26/99 (26%)

Table 11: Same measurements on two ClaudeCodeTrace subsets and their union. The qualitative findings replicate; the quantitative shape of the rescue distribution differs by prompt size in an interpretable direction.

6% of turns and is workload-shape dependent. On the substantive divergent slice where cacheblend's selective recompute engages non-trivially, the rescued output is judged non-inferior to a fresh-compute baseline on 63% of turns, well below the 85% spec gate. §6 discusses when this trade is acceptable.

6 Discussion

6.1 What this means for deployment

The headline takeaway from §5 is that cacheblend's rescue is real but not free. On Llama-3.3-70B-Instruct fp8 with our patches, the rescue is bimodal (median 76.5% across the main n=99 corpus, with 6% of turns reaching the published 95–99% peak; Table 5, Table 11) — and at T=0, severe (<0.3 token-identity to no-cache) divergence appears on 30% of substantive turns in the deep-evaluation subset (n_{sub}=47) and on 26% in the combined main corpus (n=99). On the divergent slice of the deep subset (n_{div}=19), an LLM judge prefers cacheblend's output on only 31.6% of pairs (Table 8). The asymmetry has three deployment-relevant implications.

For hit-rate-only workloads, the quality cost is irrelevant — but the latency cost still applies. Any deployment that uses prefix caching for throughput accounting without surfacing model outputs to end users — bulk embedding pipelines, throughput-benchmarking harnesses, KV-cache research itself — is unaffected by the 30% divergence rate: those outputs are not consumed. The TTFT result of §5.2 (cacheblend 55% slower at median, 7× slower at peak rescue) still applies: cache hits are not free latency on a 70B fp8 + H100 backend because LOAD bandwidth approximately cancels prefill savings. Whether cacheblend is a net win for these workloads depends on the deployment's LOAD-to-prefill cost ratio (smaller models, faster interconnect, slower compute, or throughput-vs-latency optimization all shift the answer).

For interactive coding agents like Claude Code, the divergence is load-bearing. The 37% no_cache-preferred rate is material at deployment scale: a system that selects a measurably worse next step on roughly one in three post-cache turns produces a perceptible quality regression. Worse, the failure mode is sometimes a *decode-mode flip* (tool call→prose, 11% of substantive turns); a CC harness that expects a tool-call JSON and receives prose must either retry, surface a degraded user experience, or silently fall through.

For batch RAG-style workloads, the published cacheblend evaluation remains valid. Our finding is specific to agent traffic with multi-turn cache state accumulation; RAG-style workloads where cache LOOKUP fires once against a single retrieved document set do not exhibit the same dual-mode divergence pattern. The cacheblend paper's RAG measurements [16] are not contradicted by ours.

6.2 Why agent traffic is harder than RAG for cacheblend

The cacheblend paper [16] reports near-100% output-quality preservation on permuted-RAG benchmarks at the same 0.15 recompute ratio we use. Our 60% bit-identity + 30% severe-divergence + 8.5% asymmetric mode-flip rate is a much worse quality picture on the same mechanism. The 0.15 ratio did not change; the workload did. We hypothesize three structural reasons, which we are not yet able to fully ablate but state as testable hypotheses for follow-up work:

(i) Decode mode is binary in agent traffic, continuous in RAG.. A RAG response is a prose answer where a small logit perturbation at position k produces a slightly different word at position k; the rest of the answer remains semantically close. An agent response in CC's protocol must commit, within the first ~5 output tokens, to one of two modes: emit `{"type": "function"` (tool-call) or emit natural-language prose (which the harness cannot act on). Cacheblend's selective recompute introduces a bounded logit perturbation, but at position 0 of the output that perturbation can flip the argmax from `{` to `T`/`B`/`L` (common prose openers). Once a mode is chosen, attention to the already-emitted prefix reinforces it. RAG outputs do not have this binary regime sensitivity; agent outputs do, and §5.5's 8.5% asymmetric stall rate is the direct consequence.

(ii) Multi-turn cache accumulation amplifies drift on shared chunks. RAG is typically single-turn: retrieve, blend, answer. Agent traffic is multi-turn, and shared chunks between turns are common — the system prompt, repeated tool definitions, and prior conversation history all appear in successive prompts. *In our replay setup, prompts are fixed captured request bodies; the model's generated outputs do not feed back into subsequent prompts.* The drift accumulation is purely

cache-side: turn N's STORE writes KV for the prompt chunks it just processed — including any chunks whose KV was itself produced by cacheblend's selective recompute on turn $N-1$. When turn $N+1$ LOOKUPs a chunk that overlaps with turn N's prompt, it retrieves whatever KV turn N wrote, which may already be one blending step away from fresh compute. The cacheblend paper's "small but bounded" drift bound applies to each individual blend operation; we hypothesize that on chunks shared across multiple consecutive turns the cumulative drift compounds. This is a hypothesis we have not directly isolated — a clean test would require comparing single-turn vs multi-turn replays at fixed Nth-turn cache states, which we do not run here.

(iii) Templated prompts have positional dependencies that permuted-RAG does not. The cacheblend mechanism is most accurate when the model's attention to a chunk is approximately invariant to that chunk's exact position — permuted-RAG, by construction, trains on this kind of invariance. CC's prompts are highly templated (system prompt with tool list at fixed offsets, conversation history in chronological order, current user message at a known position) and the model has learned strong position-conditioned patterns. When cacheblend re-uses a cached chunk at a new position, the chunk's KV reflects attention computed in the *original* positional context, and the selective-recompute correction sees only 15% of layers through the new positional context. For position-sensitive templates the residual drift on the un-recomputed 85% of layers compounds; for permuted-RAG it largely cancels.

Hypothesis (iii) is directly testable: at `LMCACHE_BLEND_RECOMPUTE_RATIOS=1.0`, every layer is recomputed in the new positional context, so the un-recomputed residual vanishes and identity to no_cache should recover. We test this directly in §6.3 and find it does. Hypotheses (i) and (ii) remain open empirical questions for future work.

6.3 Recompute-ratio sweep tests hypothesis (iii)

To probe hypothesis (iii) of §6.2 — residual drift on the un-recomputed layers — we sweep `LMCACHE_BLEND_RECOMPUTE_RATIOS` over $\{0.15, 0.5, 1.0\}$ on the `swebench_verified/pytest-7490` fixture (4 turns, 3 substantive at ≥256 prompt tokens, samples=1, otherwise identical to the SWE-V+`post_compact` subset methodology):

Ratio	Mean token-identity to NC	Aggregate rescue %
0.15 (default)	0.847	27.4%
0.50	0.831	27.4%
1.00	**1.000**	27.4%

Table 12: Recompute-ratio sweep on `pytest-7490` (n=3 substantive turns per ratio). Identity to the no_cache baseline recovers to 1.000 at ratio=1.0 (every layer recomputed in the new positional context) but stays around 0.83 at the intermediate ratio, not 0.92 as one would extrapolate. Rescue percent is unchanged because `RECOMPUTE_RATIOS` controls layer recompute, not which KV tokens are loaded from cache.

Two findings:

Hypothesis (iii) is supported. At ratio=1.0, identity to `no_cache` recovers bit-perfectly. Recomputing every layer in the new positional context eliminates the residual drift that the default 0.15 ratio carries. The mechanism we hypothesized in §6.2(iii) is the load-bearing one for the identity gap.

Partial recompute did not partially recover on this probe. Going from 0.15 → 0.5 (3.3× more layers recomputed) barely moves identity (0.847 → 0.831), within noise on n=3. Recovery is concentrated at the discrete → 1.0 jump on the data points we have. *We do not claim the intermediate-ratio tradeoff curve is established from n=3:* a larger sweep (denser ratio grid × the full deep-evaluation subset) is needed before concluding that no monotonic recovery exists between 0.15 and 1.0. What is robust from this probe is the discrete endpoint contrast: ratio 1.0 produces identity = 1.000 by construction (every layer recomputed in the new positional context), confirming the positional-context hypothesis. Whether 0.5 or 0.8 would offer a useful Pareto-improvement remains an open question.

The combination of §5.2's TTFT measurement (cacheblend already slower than `no_cache` at the default 0.15 ratio on our backend) and this sweep's discrete-jump pattern suggests the default 0.15 ratio is not a Pareto-optimal operating point on our backend, and ratio=1.0 eliminates the rescue benefit by recomputing every layer. A genuine graded tradeoff curve — one that gives proportional identity recovery against linear cost — likely requires mechanism changes beyond the recompute-ratio knob: candidate directions include smaller cacheable blocks, layer-importance-aware recompute selection, and model-architecture co-design.

Sample size caveat. Three substantive turns per ratio is a small sample; the 0.847 → 0.831 dip at 0.5 is plausibly noise. What is robust is the discrete jump at 1.0 from ~0.84 to exactly 1.000 — that comes from the math, not the sample. A larger probe across the full SWE-V+`post_compact` subset would tighten the intermediate-ratio identity estimates but is unlikely to change the discrete 0.15↔1.0 endpoint contrast.

6.4 Resource-constrained methodology choices

This paper was produced on a fixed compute budget ($25 envelope for the headline experiments, against a 2× H100 80GB SXM pool with real per-pod cost). Several methodological tightenings a reviewer would reasonably ask for — a raw-no-cache baseline that bypasses our parser to validate separator injection is content-neutral, an intra-condition repeatability study to bound run-to-run nondeterminism, instrumented per-phase TTFT breakdowns (LMCache LOAD vs prefill vs blend vs scheduler vs decode), a denser recompute-ratio grid, a quantitative patch ablation that runs each intermediate configuration as its own pod replay, and a bf16-vs-fp8 precision comparison — are individually small but cumulatively a substantial increase in pod-hours, well beyond this project's compute budget. We list them explicitly as open empirical questions rather than implicit ones, so that a reader who wants to commission this work knows where it sits relative to our existing claims. The headline findings (rescue is bimodal, identity gap is real, latency cost is real, mode-flip asymmetry is real) are robust to the methodology gaps we name here — they are direct read-offs from

the measurements we did run, not inferences that depend on the missing instrumentation. The *causal* explanations for those findings are where the missing instrumentation would matter most; we have flagged each one as a hypothesis rather than a conclusion.

6.5 Limitations

Single model. All measurements are on Llama-3.3-70B-Instruct fp8. The cacheblend mechanism is model-agnostic in design, but the empirical quality cost is not: the fp8 quantization noise floor, the 70B parameter count, and Llama's attention architecture all interact with cacheblend's selective recompute. The same patches and methodology on Sonnet, Haiku, Llama-8B, or Qwen could produce a different divergence distribution. §6.3's recompute-ratio probe is the higher-priority follow-up; multi-backend characterization is the longer-horizon next step.

Judge calibration. We replicate the 19-pair A/B against Opus-4.7 and Haiku-4.5 in §5.4, and the three-judge majority vote reproduces Sonnet-4.6's headline numbers (31.6% preference, 63.2% non-inferior). The single-judge risk is reduced but not eliminated — all three judges share Anthropic's RLHF training, so a coordinated calibration bias could still apply. A human spot-check is a sensible next step we do not run here.

Single decoding temperature. $T = 0$ greedy decoding makes single-token differences load-bearing: a divergence at position k permanently forks the trajectory. Sampling at $T = 0.7$ would average over multiple plausible continuations and might mask or amplify the cross-condition divergence in the marginal distribution. We defer the $T > 0$ distributional measurement to follow-up work.

Sample size. 19 divergent pairs is small. The 95% Wilson CI on the cacheblend preference rate spans [15.4%, 54.0%] — we cannot reject "cacheblend is preferred half the time" from the data alone. The point estimate's location well below 50% is what supports the non-inferiority gate failing; widening the corpus would tighten the interval but is unlikely to change the qualitative direction.

Workload diversity within ClaudeCodeTrace, but a single corpus source. The combined n=99 corpus mixes SWE-bench-Verified sessions, `post_compact/` canonical captures, and `skill_invocation/` (§5.6). All three subsets come from one researcher's laptop running one client family (CC SDK), so the captured prompt distribution is not yet broad across operators, languages, or task domains. The cross-corpus replication of the bimodal-rescue and severe-divergence patterns supports generality *within Claude-Code-shaped traffic*; we make no claim about other agent harnesses (Cursor, Aider, OpenAI Codex, etc.) on this evidence alone.

Backend specificity. All measurements use a Llama-3.3-70B-Instruct fp8 backend on 2× H100 80GB SXM, vLLM v0.19.0, and lmcache v0.4.2 with our patches. We do not claim generality across smaller models, different precisions (bf16), faster interconnects, or non-vLLM serving stacks. The bf16 vs fp8 comparison is the highest-priority open empirical question; §6.3's recompute sweep already isolates one hypothesis on the present backend.

6.6 Recommendation

We make one concrete methodological recommendation: *KV-cache reuse research should report output-quality measurements alongside hit-rate measurements, on at least one agentic workload, with a structured methodology that includes $T = 0$ token-identity and a strong-judge blind A/B preference test.* The hit-rate-only framing that the field has converged on is sufficient for the RAG-style workloads the early prefix-caching literature targeted, but it silently elides exactly the failure mode we surface here: a mechanism can hit at the published rate and still produce systematically worse output on the same input. ClaudeCodeTrace and the skillcacher harness provide the reproducible substrate for quality-aware KV-reuse evaluation on agent traffic.

7 Related Work

LLM serving infrastructure. vLLM [11] introduced paged-attention KV caching as a production-grade serving substrate; SGLang [19] extended the paged-attention idea with RadixAttention's tree-structured prefix sharing; TGI [8] and DistServe [20] explored complementary directions in batching and disaggregated prefill/decode. skillcacher layers on top of vLLM and reuses its paged-attention prefix cache without modification; our contribution is at the proxy and cross-request integration layer, not at the inference kernel.

KV cache reuse and prefix caching. The closest prior work is the LMCache + cacheblend line [14, 16], which provides the cross-request KV store and the selective-recompute mechanism that skillcacher integrates. Prompt Cache [6] addressed a similar problem from the compiler-IR direction (statically-analyzable prompt templates), and ChunkAttention [17] extended prefix-sharing to the attention computation itself. None of these prior systems target Anthropic-shaped agent traffic, and none measure the output-quality cost of their reuse mechanism on a non-RAG workload. Our contribution against this thread is twofold: (a) the patches and proxy needed to actually run cacheblend against natural CC traffic on a non-toy backend (§3.2), and (b) the quality-versus-rescue characterization in §5 that the published cacheblend evaluation did not surface.

LLM agent benchmarks. SWE-bench [9] and its curated variant SWE-bench Verified [15] provide the canonical task suite for agentic coding evaluation; MetaGPT [7] and AgentBench [13] extend the agent-benchmark surface to multi-agent and multi-domain settings. These benchmarks all measure *end-task success*: did the agent close the bug, pass the tests, write the working code. ClaudeCodeTrace is complementary, not a replacement — it measures *traffic-shaped* serving-system properties (cache hit rate, output identity, judge preference) on captures of CC sessions that themselves work on SWE-V tasks. A serving-system paper can use ClaudeCodeTrace to benchmark cache strategy; an agent-quality paper still needs SWE-V (or successor) to measure task success.

Output-quality evaluation. MT-Bench [18], AlpacaEval [12], and Chatbot Arena [5] established LLM-as-judge as a credible methodology for ranking model outputs. We apply the same methodology in a different setting — not to rank candidate *models* on the same input, but to rank the outputs of the same model under different

cache strategies. The position-randomization and prompt-caching elements of our setup follow the MT-Bench [18] recipe. We additionally run three-judge replication (Sonnet-4.6 / Opus-4.7 / Haiku-4.5; Table 9) and find that the majority-vote verdict reproduces the single-judge headline rates exactly, which is consistent with the cited works' observation that multi-judge consensus is a strict improvement over single-judge ranking but the calibration gap is small when the divergent-pair sample is well separated.

8 Conclusion

We presented skillcacher, a transparent proxy (drop-in for Claude Code clients given a patched lmcache backend) that integrates cacheblend with Claude-Code-shaped traffic replayed on a Llama-3.3-70B fp8 + vLLM + LMCache backend; ClaudeCodeTrace, a benchmark of 13 captures / 182 turns / 411k tokens of redacted on-wire CC request bodies; and, to our knowledge, the first measurement of cacheblend's agent-protocol usefulness cost on natural agent traffic at this backend scale. Across two corpus subsets (combined n=99 substantive turns) cacheblend rescues a median 76.5% of prefill tokens; the published 95–99% steady-state peak appears on only 6% of turns and disappears entirely on the `skill_invocation/` workload (Table 5, Table 11). The rescue is not free: at T=0, 30% of substantive turns produce severely divergent output (Sonnet-4.6 judge analysis on the divergent slice n_{div}=19 from the SWE-V+`post_compact` subset: 31.6% prefer cacheblend, 63.2% conditional non-inferiority versus an 85% pre-registered gate; Opus-4.7 and Haiku-4.5 majority-vote replicates Sonnet exactly; Table 8, Table 9).

Our methodological claim is independent of the patches: *hit rate is not output quality, and a KV-cache reuse mechanism that hits at the published rate while producing systematically different output on the same input is not the same artifact.* skillcacher and ClaudeCodeTrace are the substrate for asking that second question on agent-shaped workloads.

Availability. ClaudeCodeTrace is available now at https://huggingface.co/datasets/intelchen/claudecode-trace under CC-BY 4.0. The dataset contains redacted CC outbound request bodies (§4.3) collected from the lead author's own usage of Claude Code; redaction removes Tailscale/HF/RunPod credentials, local filesystem paths, and the CC client account identifier. Anthropic Claude Code is closed-source proprietary software; the redacted captures themselves are released as factual recordings of the wire protocol (analogous to network capture fixtures), which we believe is consistent with CC's terms of service for non-derivative research use, but we make no legal warranty. skillcacher source, the patched lmcache wheel, the bench harness, and the figure-generation scripts will be released on acceptance under MIT license. For double-blind review, a frozen submission-time tag is accessible via an anonymous Zenodo archive linked from the submission. The `scripts/dev/extract_full_corpus_rescue.py` and `extract_tool_call_rate.py` drivers used for the quantitative tables in §5 are part of that artifact and re-run end-to-end against the public ClaudeCodeTrace.

References

[1] Anthropic. 2025. Claude Code. https://claude.com/code.

[2] Yiheng Chen. 2026. Chunk-aligned LOOKUP fix for LMCache v0.4.2. PR pending upstream; see source at https://github.com/intelc/skillcacher-public.

[3] Yiheng Chen. 2026. ClaudeCodeTrace: A Public Benchmark of Claude Code Captures. https://huggingface.co/datasets/intelchen/claudecode-trace, CC-BY 4.0.

[4] Yiheng Chen. 2026. skillcacher: Source code, bench harness, and paper sources. https://github.com/intelc/skillcacher-public, MIT.

[5] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. 2024. Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. arXiv:2403.04132.

[6] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *MLSys 2024*.

[7] Sirui Hong et al. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. arXiv:2308.00352.

[8] Hugging Face. 2023. Text Generation Inference (TGI). https://github.com/huggingface/text-generation-inference.

[9] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *ICLR 2024*.

[10] Toru Kasai, Gunho Lee, Hiroki Arimura, Setsuo Arikawa, and Kunsoo Park. 2001. Linear-Time Longest-Common-Prefix Computation in Suffix Arrays and Its Applications. In *Combinatorial Pattern Matching (CPM)*. 181–192.

[11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. doi:10.1145/3600006.3613165

[12] Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. AlpacaEval: An Automatic Evaluator of Instruction-following Models. https://github.com/tatsu-lab/alpaca_eval.

[13] Xiao Liu et al. 2023. AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688.

[14] LMCache Team. 2024. LMCache: Cross-Request KV Cache Reuse for Large Language Model Serving. GitHub repository and design notes. https://github.com/LMCache/LMCache.

[15] OpenAI. 2024. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/.

[16] Jiayi Yao, Hanchen Li, Yuhan Liu, Siyuan Zhang, Kuntai Du, Xiaozhe Wang, Qizheng Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving with Cached Knowledge Fusion. In *EuroSys 2025*. Also: arXiv:2405.16444.

[17] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition. arXiv:2402.15220.

[18] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS 2023 Track on Datasets and Benchmarks*.

[19] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *NeurIPS 2024*.

[20] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

A Judge Prompt Template

The Sonnet-4.6 judge in §5.4 is invoked with a fixed system prompt and a per-pair user message whose A/B ordering is randomized per call. The full templates appear below verbatim; the implementation lives in `src/skillcacher/judge/prompt.py`.

System prompt.

> *You are evaluating two outputs from an AI coding agent. The agent is operating in an interactive multi-turn task where it uses tool calls (JSON function-call blocks) to read files, edit code, and make progress on software-engineering tasks.*
> *Both outputs are responses from the SAME agent context to the SAME prompt — the only difference is the engine's KV-cache strategy. Your job is to identify which output is a better next step for the agent's task.*
> *When comparing:*
> - *A well-formed tool call (JSON specifying a function name + arguments) is generally better than natural-language prose, because the agent's task requires tool calls to make progress.*

- *When both outputs are tool calls, prefer the one with more appropriate arguments — sensible file paths, reasonable command flags, useful descriptions.*
- *When both outputs are prose, prefer the one that more directly addresses the task.*
- *If the outputs are functionally equivalent (e.g., same tool with cosmetically different argument wording), respond* `EQUIVALENT`.

Respond with EXACTLY one of these tokens on the first line: `PREFER_A` / `PREFER_B` / `EQUIVALENT`. *Then on a new line, give a 1-sentence rationale (≤ 30 words).*

User template. Rendered per pair with the task prompt and both outputs substituted:

```
TASK PROMPT (what the agent was asked to do):
{task_prompt}

OUTPUT A:
{output_a}

OUTPUT B:
{output_b}
```

Parsing. Responses are parsed with the line-anchored regex `^\s*(PREFER_A|PREFER_B|EQUIVALENT)\b` applied in multiline mode. The label is mapped back to {cacheblend, no_cache, equivalent} via the recorded `position_a` for that call; anything that does not match the regex on either of the first two lines is recorded as `UNPARSEABLE` (zero occurrences across the 19-pair run). Rationales are extracted as the remainder of the response after the label token, trimmed and truncated at the first newline.

Calling conventions. The judge is invoked via the Anthropic Messages API with top-level `cache_control: {type: "ephemeral"}` on the static system prompt; per-pair user messages bypass the cache. Position assignment is seeded per pair (`random.Random(42 ^ hash((capture_id, turn_index)))`) so that re-running the judge call deterministically reproduces both the A/B ordering and any cache hits on the system prompt.

B Statistical Span Miner

The miner extracts repeated token sequences from the ClaudeCodeTrace corpus for the pre-seed dispatcher (§3.3). Inputs: a list of token streams (one per captured request). Parameters: a length floor $\ell_{\min}$=256 tokens (matching `lmcache.blend_min_tokens`), a frequency floor $f_{\min}$=3 distinct streams, a MinHash Jaccard cutoff J=0.8, and a shingle width k=5. The output is a list of `MinedSpan` records, sorted by (frequency descending, length descending), each carrying its token ids, originating stream ids, and frequency count. The reference implementation (`src/skillcacher/tagging/statistical_miner.py`, ∼340 LoC) follows the procedure below.

Empirical scale. On the union of ClaudeCodeTrace's three subsets (411k tokens, 182 streams) with $\ell_{\min} = 256$, $f_{\min} = 3$, $J = 0.8$, the miner returns **181 spans totalling 278k tokens**, in approximately 18 s on a single MacBook Pro core. The top hit is a 1,691-token span occurring in 26 of the 182 streams (the CC system prompt with tool list); the long tail of mid-frequency spans corresponds to repeated skill preambles and tool-output formatting blocks.

Algorithm 1 `mine_spans`: extract length-$\geq \ell_{\min}$, frequency-$\geq f_{\min}$ near-deduped repeated sequences from a multi-stream token corpus.

Inputs: streams $S_1, \ldots, S_m$; $\ell_{\min}$, $f_{\min}$, J, perms P, shingle width k.
Output: sorted list of mined spans.

1. *Concatenate.* Build $T = S_1 \cdot \sigma_1 \cdot S_2 \cdot \sigma_2 \cdots S_m$ where each σ_i is a unique negative sentinel that prevents suffixes from crossing stream boundaries. Maintain a parallel array `stream_of_pos` mapping each position to its originating stream id.
2. *Suffix array via prefix doubling.* Compute SA$[0 \,..\, n-1]$ such that SA$[i]$ is the starting index of the i-th lex-smallest suffix of T. Each round sorts on the composite key $(\text{rank}[i], \text{rank}[i+k])$, doubling k each iteration; terminates when all ranks are unique. Total work $O(n \log^2 n)$, memory $O(n)$. (The naive sort by full suffix string is $O(n^2)$ in expectation on this corpus and was benchmarked to hang on $n \geq$250K.)
3. *Kasai LCP.* Compute LCP$[i]$ = length of the longest common prefix of SA$[i-1]$ and SA$[i]$ in linear time via the Kasai algorithm [10].
4. *Monotonic-stack enumeration of maximal repeats.* Sweep LCP left-to-right with a stack of (height, left_index) pairs. For every entry that pops because the current LCP dips below it, emit SA[left_index$-1 \,..\, i-1$] as a candidate of length height: those suffixes share at least height characters. This enumerates the nested case (a 350-token repeat at freq=3 nested inside a 300-token repeat at freq=5 emits both).
5. *Floor + sentinel filters.* Discard candidates with length $< \ell_{\min}$, with fewer than $f_{\min}$ distinct source streams (computed via `stream_of_pos`), or containing any sentinel token (negative id). Deduplicate by fingerprint hash.
6. *Exact-prefix dedup.* Sort surviving candidates by length descending. If candidate A's tokens are a prefix of an already-kept candidate B with $|A| < |B|$ and they share at least one source stream, drop A (B is strictly more informative).
7. *MinHash near-dup dedup.* Compute a P-permutation MinHash signature over k-shingles for each remaining span. Two spans whose signature Jaccard $\geq J$ are merged, keeping the longer representative. We use P=64, k=5, J=0.8; the blake2b-keyed hash family makes the signatures deterministic across runs and tractable on 64-bit machines.
8. *Sort + return.* Sort by $(-\text{frequency}, -\text{length})$ so callers can take the top-N under a budget cap.

C Reproducibility

Pod configuration. All §5 measurements use a single pod recipe: 2× NVIDIA H100 80GB on RunPod's `SECURE` cloud, vLLM v0.19.0 with the `LMCacheConnectorV1` attached, lmcache v0.4.2 with the patches in §3.2, model `meta-llama/Llama-3.3-70B-Instruct` loaded at `dtype=fp8`, tensor-parallel degree 2, and `max_model_len=32768`. The cacheblend condition additionally exports the eight `LMCACHE_*` environment variables in Table 13; `PYTHONHASHSEED=0` is mandatory for chunk-hash stability across the connector's per-role `TokenDatabase` inits.

Provisioning resilience. RunPod's 2× H100 pool is capacity-tight enough that synchronous HTTP 500 "no instances available" rejections at `POST /pods` are routine. We combine three mitigations: (i) `CLOUD_TYPE=SECURE` with `WAIT_TIMEOUT_S=3600` and `PROVISIONING_TIMEOUT_S=1800` to let an accepted pod queue patiently for hardware; (ii) a bash retry loop around the

Variable	Value
PYTHONHASHSEED	0 (required)
VLLM_KV_CACHE_TYPE	lmcache
LMCACHE_USE_EXPERIMENTAL	True
LMCACHE_LOCAL_CPU	True
LMCACHE_MAX_LOCAL_CPU_SIZE	40
LMCACHE_CHUNK_SIZE	256
LMCACHE_ENABLE_BLENDING	True
LMCACHE_USE_LAYERWISE	True
LMCACHE_BLEND_SPECIAL_STR	" # # "
LMCACHE_BLEND_CHECK_LAYERS	1
LMCACHE_BLEND_RECOMPUTE_RATIOS	0.15 (default)

Table 13: Environment variables for the cacheblend condition. Set on the pod before `vllm serve` is launched. Omitting any of the eight `LMCACHE_*` variables causes the engine either to crash at startup or to silently fall through to plain prefix caching (no rescue).

`skillcacher-bench` invocation for synchronous create rejections (20 attempts, 120 s sleep between); (iii) sequential execution of multi-condition runs so the harness cannot multiply its own capacity demand. The `scripts/dev/recompute_probe.sh` driver in our repository is the canonical example.

Canonical commands. The complete SWE-V+`post_compact` subset replay (7 captures, 54 pairs across three conditions) is one invocation:

```
SKILLCACHER_BACKEND_MAX_COMPLETION_TOKENS=512 \
MODEL_NAME=meta-llama/Llama-3.3-70B-Instruct \
DTYPE=fp8 GPU_COUNT=2 TENSOR_PARALLEL_SIZE=2 \
MAX_MODEL_LEN=32768 CLOUD_TYPE=SECURE \
WAIT_TIMEOUT_S=3600 PROVISIONING_TIMEOUT_S=1800 \
skillcacher-bench quality-eval \
  --corpus-dir tests/fixtures/claude_code_real \
  --capture-filter \
"swebench_verified/{astropy-13398,matplotlib-26113,\
pylint-7080,pytest-7490,sphinx-11510},\
post_compact/plan4_compaction_spike_v{6_align,7_norm}" \
  --conditions no_cache,prefix_cache,cacheblend \
  --samples 1 --temperature 0 \
  --out benchmark/results/plan5_quality_section1
```

The judge analysis is then a single offline invocation reading the parquets emitted by the above:

```
export $(grep -E "^ANTHROPIC_API_KEY=" .env | xargs)
.venv/bin/python scripts/run_judge.py --skip-identical
```

Artifact availability. Three artifacts support end-to-end reproduction. (i) ClaudeCodeTrace on Hugging Face [3], available under CC-BY 4.0; the `scripts/download_claudecode_trace.py` wrapper accepts `–subset` filters. (ii) skillcacher source, including the patched lmcache wheel build script, the proxy, the bench harness, and the figure-generation scripts; the analysis drivers used for §5.1 and §5.5 (`scripts/dev/extract_full_corpus_rescue.py`, `scripts/dev/extract_tool_call_rate.py`) are part of this set. (iii) The pod-orchestration driver (`scripts/dev/oneshot_pod.{sh,py}`) and the recompute-ratio sweep driver (`scripts/dev/recompute_probe.sh`). Items (ii) and (iii) are released under MIT at [4]; the paper PDF, prebuilt figures, and this appendix's reproducibility scripts all live in the same repository at its root.

D Anchor Recognition Walkthrough

To make §3.1's parser concrete, this appendix shows one real CC request body (lightly redacted) with the structural anchors color-highlighted and the cacheblend separator " # # " marked at each injection site.

Color key. header (per-turn, normalized per §3.2.2); gitStatus ; commits ; `<system-reminder>` . The injected separator is shown as `# #` between blocks; cacheblend treats each separated span as one chunk for STORE / LOOKUP / blend.

```
# #
x-anthropic-billing-header: cc_version=NORM; cch=NORM;
# #
CWD: /Users/⟨user⟩/skillcacher
Date: 2026-05-08
gitStatus: This is the git status. . .
Current branch: main
Status:
M benchmark/results/audit/...
?? scripts/dev/cacheblend_quality.sh
# #
Recent commits:
f8a0cfa fix(capture): plant. . .
12778ec docs: Plan 3 resume. . .
# #
⟨system-reminder⟩
As you answer the user's questions. . .
⟨/system-reminder⟩
# #
```

Figure 7: One real CC request, anchors highlighted. The parser identifies four blocks: the per-turn billing header (yellow), the gitStatus block (blue), the Recent-commits block (pink), and the `<system-reminder>` wrapper (green). Cacheblend separators are injected on each side of every recognized block. The yellow block's `cc_version` and `cch` fields have already been rewritten to `NORM` by §3.2.2's header normalizer, which is why the first KV chunk's hash is stable across multi-turn sessions.

What the parser does at each anchor.

(1) **Per-turn header (yellow):** detected by the `x-anthropic-billing-header:` prefix. The `cc_version` and `cch` field

values are rewritten in place to `NORM` placeholders before chunk hashing, so the first KV chunk hash is identical across all turns of one CC session (§3.2.2).

(2) **gitStatus (blue):** detected by the literal `gitStatus:\n` prefix. Typically 100–400 tokens long; spans the working-tree summary CC injects on session start. Content is stable for the lifetime of a session.

(3) **Recent commits (pink):** detected by `Recent commits:\n`. Typically 50–200 tokens; the most recent commit hashes drift across sessions but the structural anchor is stable.

(4) **`<system-reminder>` wrapper (green):** detected by the literal tag pair. Variable-length (30–200 tokens typically), often falls below `blend_min_tokens=256` in isolation and is rolled into an adjacent block rather than getting its own separator pair — see §3.1.

Why this matters. Without the parser, cacheblend's default `blend_special_str=" # # "` never appears in CC outbound traffic — CC's wire format uses `\n\n` as its block delimiter, and the resulting 25–180-token spans fall under the 256-token floor. The parser's job is to inject the configured separator at exactly the boundaries that produce content-stable, ≥256-token chunks. The four highlighted classes above are what enables cacheblend to engage at all on natural CC traffic.